

SHOULD BE REPLACED ON REQUIRED TITLE PAGE

Instruction

1. Open needed docx template (folder "title"/<your department or bach if bachelor student>.docx).
2. Put Thesis topic, supervisor's and your name in appropriate places on both English and Russian languages.
3. Put current year (last row).
4. Convert it to "title.pdf," replace the existing one in the root folder.

Contents

1	Introduction	7
1.1	Research Questions	11
1.2	Thesis Structure	11
2	Literature Review	13
2.1	Container orchestration systems	13
2.2	Pull-based and decentralized approaches	14
2.3	Security and reliability concerns	15
2.4	Summary	16
3	Problem identification	17
3.1	Glossary	17
3.2	Business impact	18
3.3	Analysis of the existing solutions	19
3.3.1	Kubernetes	19
3.3.2	HashiCorp Nomad	23
3.4	Requirements clarification	26
3.4.1	Business Scenarios	26
3.4.2	Functional Requirements	27

3.4.3	Non-Functional Requirements	27
3.5	Requirements Finalization	28
4	Implementation	30
4.1	System Design	30
4.1.1	Pull vs. Push Models	30
4.1.2	Top-Layer Architecture	34
4.1.3	Control Plane	35
4.1.4	Data Plane	37
4.1.5	Architecture Finalization	39
4.2	Implementation	40
4.2.1	Technical Scenarios	40
5	Evaluation	51
5.1	Methodology	51
5.1.1	Test Environment	51
5.1.2	Test Scenarios	52
5.1.3	Metrics Definition	53
5.2	Test Results	53
5.2.1	Service Level Objective (SLO)	53
5.2.2	Recovery Time Objective (RTO)	54
5.2.3	Functional Validation	56
5.3	Discussion	57
5.3.1	Performance Analysis	57
5.3.2	Resource Efficiency	58
5.3.3	Design Trade-offs	58
5.3.4	Threats to Validity	59

CONTENTS	4
-----------------	----------

5.4 Conclusion	60
--------------------------	----

6 Conclusion	62
---------------------	-----------

Bibliography cited	63
---------------------------	-----------

A Extra Stuff	66
----------------------	-----------

B Even More Extra Stuff	67
--------------------------------	-----------

List of Tables

5.1	Service Level Objective (SLO) measurements with confidence intervals and percentiles	54
5.2	Recovery Time Objective (RTO) measurements with confidence intervals and percentiles	54
5.3	Control plane resource requirements for high-availability deployments. Kubernetes data from [1], Nomad data from [2].	58

List of Figures

4.1	Comparison of Push and Pull configuration propagation models. In the Push model (top), the control plane initiates outbound connections to apply changes directly to nodes. In the Pull model (bottom), each node periodically polls the control plane or a configuration repository to reconcile its local state with the desired state.	31
4.2	High-level component diagram of the proposed system. The control plane (central registry) communicates with data-plane nodes via a REST API. Each node runs a local agent that periodically pulls configuration state. An SRE interacts with the control plane to declare desired workload placements.	34
4.3	Detailed component diagram of the finalized system architecture, illustrating interactions between the SRE client, control plane (with PostgreSQL backend), data-plane nodes (each running an agent and containerized workloads), and the Edge ingress component.	39
4.4	Example workload specification in YAML format for a Python HTTP server. The <code>cpu</code> field specifies resource requirements in millicores, while <code>ram</code> is expressed in megabytes.	41

4.5	Sequence diagram illustrating the workload creation and submission workflow. The operator submits a YAML specification via mctl, which is validated and persisted by the control plane, returning a unique workload identifier.	42
4.6	Sequence diagram illustrating agent-driven workload acquisition. The agent polls for available workloads, acquires a lease, and creates the container via Docker.	44
4.7	Sequence diagram illustrating fault tolerance through lease expiration. When Agent A fails, the background task expires the stale lease after 30 seconds, enabling Agent B to acquire and reschedule the workload.	46
4.8	Sequence diagram illustrating event propagation and status inference. Agents emit lifecycle events which are persisted in PostgreSQL; the control plane infers workload status from event history patterns.	47
4.9	Comprehensive end-to-end sequence demonstrating the complete pull-based orchestration lifecycle: workload creation, agent acquisition, normal operation with lease renewal, node failure detection through lease expiration, and automatic recovery by another agent. The annotated RTO box shows how the 90-second Recovery Time Objective is achieved.	50
5.1	SLO and RTO distribution comparison. Left: SLO for single and concurrent workload scenarios. Right: RTO distribution for node failure recovery. Individual data points are shown as scatter overlay.	55

5.2 Cumulative Distribution Function (CDF) of SLO and RTO measurements. The CDF shows the probability that a measurement falls at or below a given time value.	56
--	----

Abstract

Managing cloud applications is hard. In order to deal with them, businesses use orchestration platforms. However, orchestration platforms often come with great overhead and high expenses in operating and maintaining it. To overcome related issue, our thesis aims to build a working prototype of a distributed workloads management system based on a pull-model, which will bring no overhead and be easy in deployment and operation.

Chapter 1

Introduction

Modern software systems in the cloud progressively tend to be built as reliable, scalable, and highly available. As more IT businesses adopt cloud-native practices and rely on distributed applications, it becomes essential to have a system that enables continuous monitoring, maintaining, and delivering new systems to production. The vast majority of organizations rely on tools such as Kubernetes [1] or HashiCorp Nomad [2], but these often come with high operational costs and require comprehensive technical expertise to maintain them. As an example, to setup a Kubernetes cluster, a company needs to have at least 4 CPU cores and 16 gigabytes of RAM for the control-plane component, and even then, managing upgrades, configuring networking and storage, handling security policies, and troubleshooting failures can be highly complex, often requiring specialized DevOps teams and extensive ongoing maintenance.

The *orchestration principle* [3], [4] underlies the aforementioned platforms. Recent advances in self-healing systems [5] and zero-trust security models [6] provide additional mechanisms to enhance system reliability and security. Orchestration is the process of automated management, coordination, and deploy-

ment of multiple services and components in distributed systems to ensure their consistent and efficient operation. Furthermore, due to the nature of the modern orchestration platforms, it becomes problematic to modify them and add new features. As an example, some platforms rely on the push-based model, which could lead to a certain amount of problems like: breaking delivery after network connectivity errors, or workload duplication. For businesses, it could be critical to deal with these issues, since any kind of failure on the infrastructure could potentially lead to high financial losses.

The complexity of orchestration platforms, and potential issues with maintainability and workloads management, create significant barrier for organizations, that cannot afford large DevOps departments or teams. Their setup and maintenance require not only hardware resources but also constant attention from skilled engineers. Furthermore, the reliance on a push-based model limits fault tolerance and creates bottlenecks in distributed scheduling. This thesis addresses related limitations by exploring a pull-based approach [7] to distributed workload management. The key goal of the pull-based approach is not to directly order another service to perform a task, but to place this task in some registry, from which this service will later receive the task and perform it. So, in practice instead of pushing tasks from a central system, worker nodes independently pull their configuration and workload data from the control plane. This model aims to increase reliability, simplify maintenance, and make distributed orchestration more accessible.

The purpose of this work is designing and implementing a working prototype of a distributed workloads management system with *pull-model approach*. Key objectives are: reducing operational overhead, designing decentralized architecture with ability to scale, improving security by automating manual infrastructure

operations and creating a lightweight flexible alternative to existing orchestration platforms. During each phase, artifacts will be collected, so the solution could be replicated.

To achieve the proposed goals, we aim to apply the Design Science Research [8] methodology, which mainly focuses on building and evaluating artifacts to solve real-world problems. This approach emphasizes iterative development, where each stage informs and refines the next, ensuring that the resulting system effectively addresses the identified challenges. The work is separated into several stages:

1. Problem identification and requirements clarification – analyze existing orchestration solutions, identify their limitations, and define the functional and non-functional requirements for the proposed system.
2. System design and prototype development – develop the architecture and implement a working prototype of the distributed workload management system, incorporating features like task scheduling, fault tolerance, and scalability.
3. Validation and evaluation – test the prototype under controlled and realistic conditions, measure its performance, reliability, and resource efficiency, and compare it against existing platforms to assess improvements and identify areas for further refinement.

This methodology ensures a structured, evidence-based approach to designing and assessing the proposed system while keeping practical applicability as a central focus.

It should be explicitly noted that generative artificial intelligence (GenAI) tools were used in this research solely as a language and editing aid. Specifically, GenAI assisted in refining phrasing, improving grammatical correctness, and enhancing the academic tone of the exposition. It was employed to challenge initial formulations, suggest clearer syntactic structures, and ensure terminological consistency particularly during the drafting and revision of descriptive and expository sections. However, no analytical, conceptual, architectural, or evaluative work was delegated to or generated by AI systems. All problem formulation, requirements engineering, system design decisions, implementation logic, experimental validation, and scholarly interpretation were conducted exclusively by the author. The intellectual contribution, critical reasoning, and technical insights presented in this dissertation are entirely human-derived.

The expected outcome of this research is a lightweight and reliable orchestration system prototype that demonstrates the practical benefits of applying a pull-based model in distributed workload management. The developed system is expected to reduce operational overhead by automating configuration delivery and minimizing manual interventions. It will introduce a decentralized control plane that relies on REST API and PostgreSQL for efficient coordination and data storage. Additionally, the work aims to provide a clear comparison with existing orchestration platforms, highlighting improvements in scalability, fault tolerance, and resource efficiency. As a result, this thesis contributes to simplifying infrastructure management and offers an alternative orchestration approach that can be more easily adopted by organizations with different operational scales.

1.1 Research Questions

This thesis seeks to answer the following research questions:

- RQ1.** How can a pull-based orchestration approach reduce operational complexity and resource overhead compared to traditional push-based platforms like Kubernetes, while maintaining comparable fault tolerance and self-healing capabilities?
- RQ2.** What are the fundamental architectural trade-offs between pull-based and push-based orchestration models in distributed systems, particularly with respect to consistency guarantees, coordination overhead, and fault isolation mechanisms?

1.2 Thesis Structure

The remainder of this thesis is organized as follows:

- Chapter 1:** Motivates the problem, states the research questions, outlines the **Introduction** Design Science Research methodology, and provides an overview of the thesis structure.
- Chapter 2:** Analyzes the business impact of orchestration failures, examines **Problem** existing solutions (Kubernetes, HashiCorp Nomad), and derives **Identifica-** functional and non-functional requirements for the proposed system.
tion
- Chapter 3:** Investigate modern container orchestration systems and decentral-
Literature ized, pull-based approaches in distributed computing, highlighting
Review

gaps in current research and practice.

- Chapter 4:** Presents the system architecture, design decisions, and technical implementation details through key scenarios, demonstrating how the pull-based model operates in practice.
- Chapter 5:** Evaluates the prototype against the established requirements, measures performance metrics (including RTO achievement), and compares the approach with existing orchestration platforms.
- Chapter 6:** Summarizes the contributions, limitations, and practical implications of the research, and outlines directions for future work.

Chapter 2

Literature Review

Several literature sources were employed to perform analysis of the problem. Google Scholar and Research Gate served to perform the research on the distributed compute management and pull-model approach. The keywords used to find the sources were "cloud orchestration", "distributed cloud computing", "pull-model". To fetch more practical insights we decided to use official Kubernetes, HashiCorp Nomad and Apache OpenServerless documentation.

2.1 Container orchestration systems

Modern orchestration platforms, such as Kubernetes and HashiCorp Nomad, have become standard tools for managing distributed workloads in cloud environments. Kubernetes provides a declarative configuration model and uses a centralized control plane to manage containerized workloads across multiple nodes [1]. However, it relies on hybrid (both push and pull models) control mechanism. To decide where to schedule workloads, there is a push-based mechanism, located on the control plane, which could server as another point of failure. But

for configuration delivery it uses pull model, node agent watches for changes and applies it when there are some. Nomad offers a similar architecture but emphasizes simplicity and flexibility in scheduling non-containerized workloads [2]. Nevertheless agent or client in terms of Nomad acts like a reporter of a node current state whether a resource operator. In conclusion both rely primarily on a push-based control mechanism, where the control plane assigns tasks to worker nodes. While this approach ensures strong control and consistency, it introduces higher coupling, operational complexity, and reduced fault tolerance.

Similarly, research on service mesh architectures [9] demonstrates that eliminating sidecar proxies through novel control plane designs can significantly reduce control plane overhead compared to traditional approaches like Istio. This sidecar-free architecture for multi-tenant environments provides a viable alternative to heavily-layered orchestration systems, aligning with the lightweight design principles explored in this thesis.

Research by Sebrechts et al. [3] highlights challenges in distributed application management, emphasizing the need for scalable and adaptive orchestration systems. It offers great alternative - distributing operational agents. Similarly, Pantazoglou et al. [10] propose decentralized workload management approaches to improve energy efficiency and reduce coordination overhead in large-scale systems. These studies suggest that centralization, while powerful, can become a bottleneck for resilience and scalability.

2.2 Pull-based and decentralized approaches

The pull-based paradigm, originally explored in software engineering and version control systems, introduces an alternative model for coordination and

synchronization. Godin et al. [7] examined the pull-based software development model and demonstrated its advantages in reducing coordination conflicts and improving modularity. These principles can be adapted to workload orchestration, where worker nodes autonomously retrieve workloads and configurations, thus minimizing reliance on a central scheduler.

Recent research on decentralized cloud computing supports this shift. Pantazoglou et al. [10] and Godin et al. [7] highlight that decentralized and eventually consistent systems can maintain reliability while reducing latency and operational risk. These approaches align with modern distributed architectures that prioritize resilience and fault isolation over strict central coordination.

Comprehensive surveys of geo-distributed task scheduling [11] further emphasize that distributed systems operating across heterogeneous network conditions and region-specific resource constraints require sophisticated coordination approaches. Such systems must balance performance enhancement, fairness assurance, and fault tolerance improvement while managing challenges inherent to geographic distribution, including varying network latency and computational capabilities across locations.

2.3 Security and reliability concerns

Infrastructure reliability and security are critical for large-scale deployments. Google Cloud’s production systems rely on strong security models and automated workload verification to ensure service integrity [12]. However, such designs also depend on highly centralized control planes, which can create single points of failure. Research on resilient cloud systems emphasizes the need for autonomous components that can recover independently [13] and sustain partial system failures

[14], [15]. Finally, compliance and governance are integral to reliable infrastructure, especially in enterprise and regulated environments. Emerging research also highlights the potential of decentralized and federated cloud management frameworks to enhance resilience and security while reducing reliance on single points of failure.

2.4 Summary

The reviewed literature provides valuable information on modern approaches to workload management, orchestration and security in cloud environments. Existing systems such as Kubernetes and Nomad demonstrate effective solutions but still rely heavily on centralized management models. Research shows that decentralized and pull-based mechanisms can increase scalability, resilience and autonomy. Overall, while much can be gained from the work already done, there is still considerable room for improvement - especially in developing more adaptive, fault-tolerant and secure orchestration architectures.

Chapter 3

Problem identification

3.1 Glossary

Recovery Time Objective (RTO) – a formal metric defined within business continuity and disaster recovery planning that specifies the maximum acceptable duration a system, application, or service may remain unavailable after a disruption before significant business impact occurs. RTO directly influences architectural decisions regarding redundancy, failover strategies, and automation levels; a lower RTO typically necessitates more sophisticated and costly resilience mechanisms.

Requests Per Second (RPS) – a performance indicator quantifying the number of incoming service requests a system processes in one second. RPS serves as a key benchmark for evaluating throughput, scalability, and load handling capacity, particularly in web-based and API driven applications. It is often used in conjunction with latency and error rate metrics to assess overall system efficacy under load.

Service-Level Agreement (SLA) – a contractually binding commitment

between a service provider and a client that defines the expected level of service, including measurable performance criteria such as uptime percentage, response time, RTO, and support responsiveness. Violations of SLA terms may incur financial penalties or service credits, making SLA adherence a critical driver of system design and operational practices in commercial and enterprise contexts.

3.2 Business impact

To illustrate the practical significance of workload orchestration in real-world systems, consider a business scenario involving a B2B authentication-as-a-service provider. Such a company operates an API service responsible for authenticating incoming client requests and determining access permissions. Upon initial deployment, the service handles a baseline load of 10 RPS. In a conventional deployment model, the API is hosted on a single virtual machine (VM). However, this approach introduces a critical single point of failure: if the underlying physical server hosting the VM fails, the entire service becomes unavailable, resulting in service downtime.

In the example described, the failure goes unnoticed for approximately one minute. Engineering staff then manually migrate the service to another VM, resulting in a RTO of five minutes. During this outage window, the system fails to process 3,000 authentication requests (calculated as $10 \text{ RPS} \times 300 \text{ seconds}$). Such service interruptions inevitably lead to both reputational damage and direct financial losses, particularly in B2B contexts where SLA often stipulate strict uptime guarantees and penalties for non-compliance.

In contrast, had the service been deployed within an orchestrated infrastructure such as one managed by Kubernetes or another container orchestration

platform the failure scenario would unfold differently. Upon the VM's failure, the orchestration system would automatically detect the loss of the service instance and reschedule it onto a healthy node within the cluster. Assuming a detection latency of one minute and a recovery time of 30 seconds, the total RTO would be reduced to 90 seconds. Consequently, only 900 requests ($10 \text{ RPS} \times 90 \text{ seconds}$) would be lost a 70% reduction in missed requests compared to the non-orchestrated scenario.

This example underscores the critical role of orchestration in enhancing system resilience, minimizing recovery times, and maintaining service continuity. In contemporary distributed systems, where availability and reliability are paramount, orchestration is not merely a convenience but a foundational requirement for operational robustness and business sustainability.

3.3 Analysis of the existing solutions

3.3.1 Kubernetes

Kubernetes - also known as K8s, is an open source system for automating deployment, scaling, and management of containerized applications [1].

K8s has become the de facto standard for container orchestration in modern cloud-native environments. Originally inspired by Google's internal Borg system a large-scale cluster management platform developed over a decade of operational experience Kubernetes was open-sourced by Google in 2014 and subsequently donated to the Cloud Native Computing Foundation (CNCF) in 2015 [16]. This strategic move catalyzed widespread community adoption, ensured vendor neutrality, and established a sustainable governance model that continues

to drive its evolution. As of early 2026, the official Kubernetes GitHub repository¹ boasts over 120,000 stars and contributions from more than 4,000 individual developers, reflecting its robust ecosystem and strong industry endorsement.

Today, Kubernetes extends far beyond basic workload orchestration. It provides a comprehensive platform for automating deployment, scaling, service discovery, load balancing, secrets management, rolling updates, and self-healing effectively serving as a foundational layer for microservices architectures, serverless frameworks, and GitOps-based CI/CD pipelines [17]. Its extensibility via Custom Resource Definitions (CRDs) and operators enables domain-specific automation, further cementing its role as an infrastructure control plane.

However, despite its extensive capabilities and broad adoption, Kubernetes introduces notable operational and economic challenges that warrant careful consideration particularly for small-to-medium enterprises or resource-constrained environments. Two primary concerns are examined below.

Infrastructure Cost and Resource Overhead

Running a production-grade, highly available (HA) Kubernetes cluster entails significant infrastructure overhead. The control plane components such as the API server, scheduler, and controller manager must be replicated for fault tolerance, while the distributed key-value store etcd (which maintains cluster state) requires its own dedicated and co-located resources to ensure consistency and low-latency access.

A minimally viable HA configuration typically demands:

- **Three** virtual machines (VMs) with at least 2 vCPUs and 2 GB RAM each

¹<https://github.com/kubernetes/kubernetes>

for the Kubernetes control plane;

- **Three** additional VMs (often collocated with the control plane but ideally isolated for performance and security) with 2 vCPUs and 4 GB RAM each to host an HA etcd cluster;
- **At least one** worker node with 1 vCPU and 512 MB RAM to schedule application workloads.

This baseline setup consumes approximately 15 vCPUs and 22.5 GB RAM of provisioned resources, yet yields only 1 vCPU and 512 MB RAM of net usable capacity for actual business applications a resource efficiency ratio of less than 7%. While lightweight Kubernetes distributions such as K3s [18] and K0s [19] aim to reduce this footprint by consolidating components and trimming non-essential features, they often sacrifice advanced capabilities (e.g. full RBAC granularity, audit logging, or native support for certain CSI drivers) or introduce trade-offs in security and observability. Consequently, organizations must weigh the benefits of Kubernetes feature richness against its operational footprint, especially when workloads are modest or latency-sensitive.

Operational Complexity and Expertise Requirements

Kubernetes is renowned for its steep learning curve and inherent architectural complexity. Its declarative model, while powerful, requires a deep understanding of concepts such as pods, services, ingress controllers, network policies, persistent volumes, and reconciliation loops. Misconfigurations particularly in networking, resource quotas, or security contexts are common and can lead to outages, performance degradation, or security vulnerabilities.

Maintaining a production Kubernetes cluster reliably demands specialized expertise. Industry surveys indicate that organizations typically employ at least one full-time Site Reliability Engineer (SRE) or platform engineer per Kubernetes cluster in mission-critical environments. These professionals must not only manage day-to-day operations but also implement monitoring (e.g. via Prometheus and Grafana), logging (e.g. EFK or Loki stacks), backup/restore strategies, and compliance controls. The scarcity of qualified Kubernetes specialists significantly elevates operational expenditures, particularly in the form of personnel costs. In the Russian market, Site Reliability Engineers (SREs) with proficiency in Kubernetes command monthly salaries ranging from approximately 240,000 to 280,000 RUB (net), with the national average reported at 229,000 RUB in 2025 [20]. These figures reflect a highly competitive labor market, especially in metropolitan hubs such as Moscow, where compensation approaches the upper end of this range. Internationally, the cost burden is even more pronounced: in North America and Western Europe, average annual SRE salaries exceed \$130,000–\$160,000 as of 2025 [21]. Consequently, maintaining an in-house Kubernetes platform often necessitates substantial investment not only in infrastructure but also in specialized human capital a barrier that disproportionately affects smaller organizations and startups with constrained budgets.

This complexity has spurred the growth of managed Kubernetes offerings (e.g. Amazon EKS, Google GKE, Azure AKS), which offload control plane management to cloud providers. While these services reduce operational burden, they introduce vendor lock-in risks and often shift rather than eliminate costs to the infrastructure and networking layers.

3.3.2 HashiCorp Nomad

HashiCorp Nomad is an open-source workload orchestrator designed to simplify the deployment and management of containerized, virtualized, and batch applications across distributed infrastructure [2]. Originally released by HashiCorp in 2015, Nomad positions itself as a simpler, more lightweight alternative to Kubernetes, emphasizing ease of operation and flexibility in workload types. While it lacks the overwhelming market dominance of Kubernetes, Nomad has established a dedicated user base particularly among organizations seeking a less complex orchestration solution. As of early 2026, the official Nomad GitHub repository² reflects a mature though smaller-scale project within the HashiCorp ecosystem.

Today, Nomad provides a versatile platform for orchestrating diverse workloads including Docker containers, raw executables, Java applications, and QEMU-based virtual machines. Its architecture centers on a single binary that handles both scheduling and execution, eliminating the need for external coordination services such as etcd. Nomad integrates natively with other HashiCorp tools most notably Consul for service discovery and Vault for secrets management forming a cohesive infrastructure automation stack [2]. This integration, combined with support for multi-region federation and heterogeneous workloads, has made Nomad particularly attractive for edge computing scenarios and hybrid cloud environments where simplicity and resource efficiency are paramount.

However, despite its architectural simplicity and operational advantages, Nomad presents certain limitations and concerns that warrant careful evaluation. Two primary areas are examined below: infrastructure overhead and operational complexity relative to the proposed pull-based approach.

²<https://github.com/hashicorp/nomad>

Infrastructure Cost and Resource Overhead

Nomad is explicitly designed to minimize infrastructure footprint, and in this regard it succeeds significantly better than Kubernetes. The control plane consists of a single binary approximately 80-100 MB in size that combines the API server, scheduler, and state management. A minimal high-availability Nomad cluster typically requires:

- **Three** server nodes with 1 vCPU and 512 MB RAM each to form a Raft quorum for state consensus;
- **No additional infrastructure** for coordination storage, as state is embedded within the server binary via Raft;
- **Client nodes** with base memory overhead of approximately 50-100 MB per agent, compared to Kubernetes kubelet requirements of several hundred megabytes.

This baseline configuration consumes roughly 3 vCPUs and 1.5-2.25 GB RAM for the control plane, representing a resource efficiency ratio substantially superior to Kubernetes. Real-world deployments report that Nomad-based clusters can reduce server costs by approximately 20% compared to equivalent Kubernetes setups [2]. In resource-constrained environments such as edge devices or IoT gateways, this difference becomes critical: one documented case demonstrated battery life extending from 6 months with Kubernetes to 18 months with Nomad a threefold improvement attributable solely to reduced overhead [2].

Operational Complexity and Architectural Concerns

Nomad's primary value proposition lies in its operational simplicity relative to Kubernetes. The learning curve for basic Nomad operations typically requires approximately two weeks, compared to two months or more for production-grade Kubernetes proficiency. Configuration uses HashiCorp Configuration Language (HCL) which is generally more concise than Kubernetes YAML. However, Nomad retains a fundamentally push-based scheduling model: the central control plane evaluates job submissions and proactively pushes allocations to client nodes, rather than having agents pull work on-demand [2]. While this approach is simpler than Kubernetes multi-component architecture, it still requires the control plane to maintain cluster-wide state and make centralized scheduling decisions.

Additionally, Nomad faces significant ecosystem and licensing challenges. The community and third-party tooling around Nomad remain substantially smaller than Kubernetes ecosystem. Fewer monitoring integrations, CI/CD plugins, and commercial support options exist for Nomad. More critically, in August 2023 HashiCorp transitioned Nomad from the Mozilla Public License (MPL) 2.0 to the Business Source License (BSL) 1.1 a move that sparked controversy and raised concerns about vendor lock-in. Unlike Kubernetes which operates under the permissive Apache 2.0 license and is governed by the vendor-neutral Cloud Native Computing Foundation, Nomad is now owned by IBM following its \$6.4 billion acquisition of HashiCorp in 2024. This combination of a restrictive source-available license and corporate ownership introduces long-term adoption risks for organizations requiring genuinely open-source infrastructure components with sustainable community governance.

3.4 Requirements clarification

Following a comprehensive competitive analysis and a thorough understanding of business imperatives, the next phase involves the systematic elicitation and formalization of requirements for the proposed orchestration prototype. To ensure alignment between stakeholder expectations and technical implementation, requirements are categorized the following way: business scenarios (representing business and operational perspectives), functional requirements (defining system capabilities), and non-functional requirements (specifying quality attributes and constraints).

3.4.1 Business Scenarios

The business scenarios were represented as user stories. No stakeholder interviews were conducted for this phase; instead, decisions were based on prior analysis. The following user stories were derived from that analysis and capture needs across business leadership, technical management, and operational engineering roles. These narratives were identified as central to the system's value proposition.

- *As a business owner, I want IT services to experience minimal or no downtime so that customer trust and revenue streams remain uninterrupted.*
- *As a Chief Technology Officer (CTO), I want infrastructure failures not to result in significant operational or financial losses.*
- *As a Site Reliability Engineer (SRE), I want the failure of a base infrastructure component such as a virtual machine to trigger automatic remediation without manual intervention.*

- *As a SRE, I want application workloads to be automatically rescheduled onto healthy, available nodes upon VM failure.*
- *As a SRE, I want a unified ingress point (API gateway) for all externally exposed applications. For the prototype scope, supported protocols shall be HTTP-based: RESTful APIs, gRPC over HTTP/2, and GraphQL.*

3.4.2 Functional Requirements

Derived directly from the above user stories, the following functional requirements specify the system's mandatory behaviors:

- FR1.** The system shall be distributed and fault-tolerant, ensuring continued operation despite the failure of individual compute nodes.
- FR2.** The system shall automatically detect node (e.g., VM) failures and reschedule affected workloads onto healthy nodes within the cluster.
- FR3.** The system shall provide a unified ingress layer (API gateway) capable of routing external traffic to internal services based on protocol and path, with initial support limited to HTTP-based protocols: REST, gRPC (via HTTP/2), and GraphQL.

3.4.3 Non-Functional Requirements

To ensure the system meets real-world operational standards, the following non-functional requirements critical for reliability, security, and maintainability were established:

- NFR1. Recovery Time:** Workload rescheduling following a node failure must complete within **90 seconds** (inclusive of failure detection and service restoration).
- NFR2. Control Plane Resilience:** The system shall remain operational and capable of managing workloads even during transient or partial outages of the orchestration control plane (e.g., through decentralized scheduling or edge-based failover mechanisms).
- NFR3. Scalability:** The architecture shall support horizontal scaling of both control and data planes to accommodate future growth in workload volume and cluster size without architectural refactoring.
- NFR4. Security:** All inter-component communication shall be encrypted in transit using Transport Layer Security (TLS) 1.3 or higher. Mutual TLS (mTLS) shall be enforced for service-to-service communication within the cluster to ensure authentication and data integrity.

3.5 Requirements Finalization

Taken together, these requirements reflect a deliberate balance between operational pragmatism, business continuity, and technical feasibility. They establish a clear design envelope for the prototype: a lightweight, self-healing orchestration layer that prioritizes rapid recovery, protocol-aware ingress, and secure communication while explicitly avoiding the resource overhead and administrative complexity associated with full-scale Kubernetes deployments. The 90-second RTO serves as a measurable benchmark for validation, and the emphasis on control plane resilience addresses a key gap in existing lightweight orchestration

solutions. Ultimately, this requirements baseline not only encodes stakeholder expectations but also provides a testable foundation against which architectural alternatives and implementation outcomes can be rigorously evaluated.

Chapter 4

Implementation

4.1 System Design

4.1.1 Pull vs. Push Models

A fundamental architectural decision in distributed orchestration systems concerns the mechanism by which configuration updates and workload instructions are propagated from the control plane to worker nodes. Two predominant paradigms exist: the Push model and the Pull model. These differ not only in data flow direction but also in their underlying philosophy imperative execution versus declarative reconciliation with significant implications for scalability, resilience, and operational complexity.

The Push model operates imperatively: a central controller initiates synchronous, outbound connections to target nodes to enforce configuration changes immediately upon request. Tools such as Ansible exemplify this approach, often leveraging secure shell (SSH) or Windows Remote Management (WinRM) without requiring persistent agents.

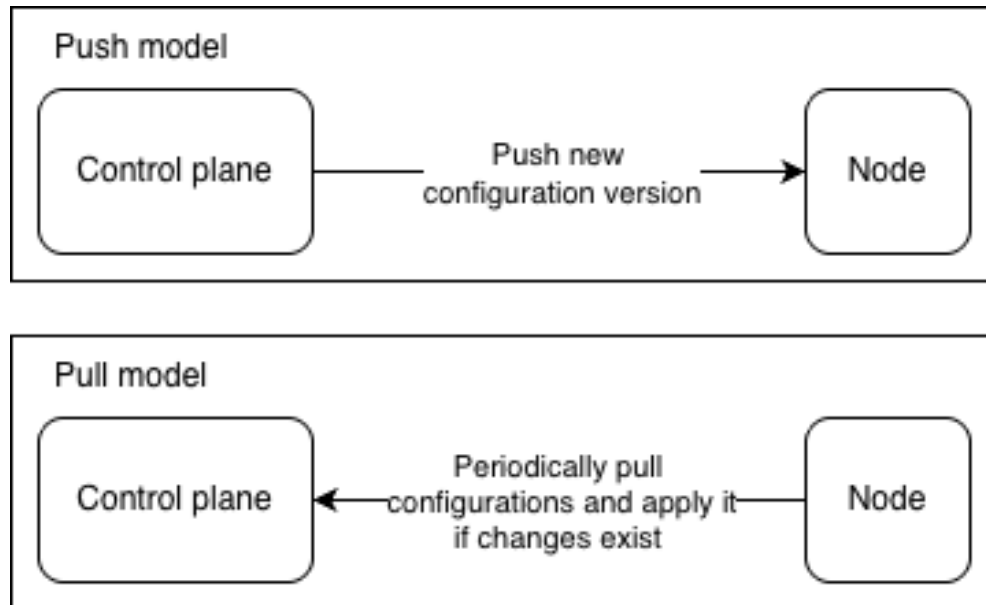


Fig. 4.1. Comparison of Push and Pull configuration propagation models. In the Push model (top), the control plane initiates outbound connections to apply changes directly to nodes. In the Pull model (bottom), each node periodically polls the control plane or a configuration repository to reconcile its local state with the desired state.

Key advantages include:

- **Immediate enforcement:** changes are applied with minimal delay once triggered by an operator.
- **Predictable timing:** the operator retains full control over when a change is deployed.
- **Simplified auditing:** all modifications originate from a single, identifiable source.
- **Reduced node-side footprint:** agentless implementations (e.g., Ansible) avoid long-running processes on worker nodes.

However, this model presents notable limitations in large-scale or dynamic environments:

- **Scalability bottlenecks:** the central controller becomes a performance bottleneck when managing thousands of nodes concurrently.
- **Network constraints:** requires bidirectional connectivity from the controller to every node problematic in environments with firewalls, NAT, or private subnets.
- **State drift vulnerability:** nodes that experience local configuration changes or reboots remain out of sync until the next manual or scheduled push.
- **Expanded attack surface:** the controller must securely authenticate to all nodes, often necessitating complex credential or key management.

In contrast, the Pull model is inherently declarative: each node runs a lightweight agent that periodically (e.g., every 10 seconds) queries a central configuration source such as an API server or Git repository and autonomously reconciles its local state with the declared desired state.

This approach offers several critical benefits aligned with modern reliability engineering principles:

- **Self-healing:** nodes automatically recover from configuration drift or transient failures without external intervention.
- **Horizontal scalability:** load is distributed across nodes; the control plane serves read requests rather than initiating writes.
- **Network resilience:** nodes only require outbound connectivity (e.g., HTTPS to a config server), making the model compatible with restrictive network topologies.

- **Decoupled architecture:** failures in the control plane do not immediately halt node operations; nodes continue functioning with their last known good state.

Nonetheless, the Pull model introduces its own trade-offs:

- **Enforcement latency:** configuration changes are applied only at the next polling interval, unless supplemented with event-driven triggers (e.g., web-hooks).
- **Agent overhead:** requires a persistent process on each node, increasing resource consumption and attack surface at the edge.
- **Temporal inconsistency:** nodes may operate with slightly divergent configurations until synchronization occurs.
- **Observability complexity:** determining the exact time of state convergence across the fleet requires additional telemetry.

Upon thorough analysis, it becomes evident that the requirements established in Section 3.5 particularly the 90-second Recovery Time Objective (RTO), resilience to infrastructure failures, and support for autonomous recovery align closely with the core strengths of the Pull model. The capacity for self-healing, network resilience, and decentralized operation directly addresses the business imperatives of minimizing downtime and reducing manual intervention.

Consequently, the Pull model has been selected as the foundational architectural paradigm for the proposed prototype. Furthermore, the design will incorporate mitigations to address its inherent limitations:

A configurable, short polling interval to bound recovery latency; Optional event-based push notifications to trigger immediate reconciliation when critical

updates occur; Minimalist agent design to reduce resource footprint; Centralized telemetry to monitor configuration convergence across the cluster. This hybrid-aware Pull approach ensures adherence to non-functional requirements while preserving operational simplicity and scalability.

4.1.2 Top-Layer Architecture

Given the explicit non-functional requirements for decentralization, fault tolerance, and automated recovery, the system must be designed from the ground up as a distributed architecture. Modern infrastructure platforms typically decompose into two orthogonal domains: the control plane and the data plane. This separation enables scalability, operational clarity, and independent evolution of concerns.

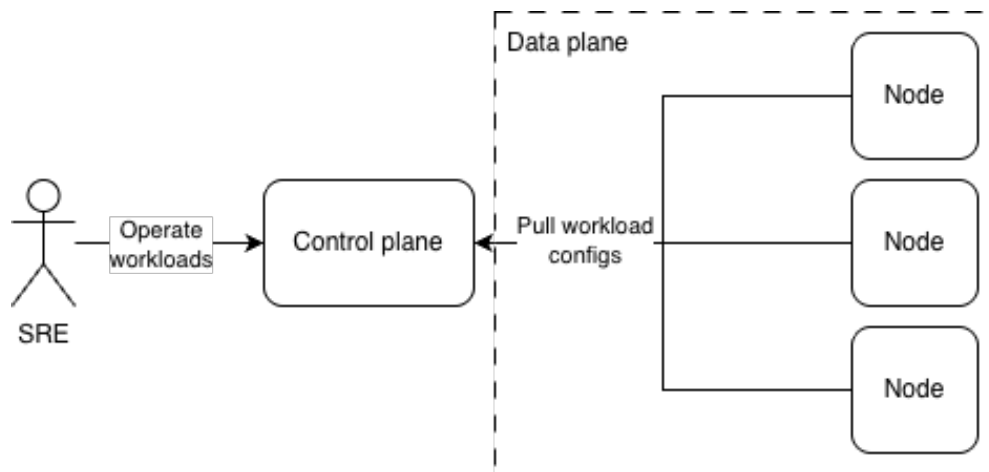


Fig. 4.2. High-level component diagram of the proposed system. The control plane (central registry) communicates with data-plane nodes via a REST API. Each node runs a local agent that periodically pulls configuration state. An SRE interacts with the control plane to declare desired workload placements.

The control plane serves as the authoritative source of truth for the desired system state. It maintains metadata about workloads (e.g., placement rules,

resource constraints, health policies) and exposes interfaces for human (e.g., SRE) and programmatic interaction. Critically, it does not execute user workloads but orchestrates their placement and lifecycle.

The data plane consists of a fleet of homogeneous infrastructure units - virtual machines, that host and execute actual workloads. These units are treated as cattle, not pets: individually ephemeral and replaceable, but collectively resilient [22].

This clean separation ensures that failures in the data plane do not compromise global coordination, while control-plane resilience mechanisms (discussed below) prevent single points of failure.

4.1.3 Control Plane

The control plane is a mission-critical component: without it, new workloads cannot be scheduled, and failed workloads cannot be rescheduled. To meet the requirement of resilience to control-plane downtime (NFR2), the control plane must be designed as stateless in its compute layer, enabling horizontal scaling and active-active replication across multiple instances.

Persistent Storage

A durable, strongly consistent datastore is required to persist the desired state of the system. After evaluation of alternatives (including etcd, Consul, and embedded SQLite), PostgreSQL was selected as the primary storage backend due to the following advantages: open-source and permissively licensed, ensuring cost efficiency and freedom from vendor lock-in; mature, actively maintained, with a large community and enterprise-grade tooling; ACID-compliant, providing strong consistency guarantees essential for orchestration correctness; widely adopted in

production systems, simplifying operational expertise and troubleshooting.

Given the expected workload primarily metadata operations (create, read, update of service definitions) PostgreSQL is more than sufficient in terms of throughput and latency. If scaling limits be approached (e.g., >10k operations/second), the architecture supports migration to PostgreSQL-compatible distributed databases such as YugabyteDB or CockroachDB with minimal application-level changes.

Application Interface

To enable communication between the control plane and data-plane agents, a well-defined RESTful API is exposed over HTTPS. While alternatives like gRPC offer superior performance and streaming capabilities, REST was chosen for the following reasons: simplicity of implementation and debugging during the prototype phase; broad tooling support (curl, Postman, standard HTTP clients); sufficient adequacy for the Pull model, where agents perform periodic, low-frequency polling (e.g., every 30 seconds); ease of integration with future API gateway extensions (e.g., authentication, rate limiting).

All technical API endpoints enforce mutual TLS (mTLS) to ensure confidentiality and authenticity of control-plane communications.

Ingress

Since workloads may be scheduled on any node within the data plane, services that require external exposure via HTTP endpoints necessitate a unified ingress point to ensure consistent and discoverable access. To fulfill this requirement, a new dedicated component (henceforth referred to as the Edge) is

introduced. The Edge component continuously discovers active workloads that declare HTTP-based network exposure (e.g., via annotations or configuration metadata) and dynamically configures routing rules to proxy incoming requests to the appropriate backend instances over a single, canonical endpoint.

The Edge operates as a lightweight, programmable reverse proxy integrated into the orchestration control plane. While production-grade alternatives such as NGINX or Envoy could serve as drop-in replacements for the Edge component, their adoption would require additional engineering effort to implement real-time synchronization of downstream service endpoints typically achieved through integration with service discovery mechanisms or dynamic configuration APIs (e.g., Envoy's xDS protocol). For the scope of this prototype, a minimal custom implementation is preferred to maintain architectural simplicity and reduce external dependencies, while retaining the flexibility to migrate to standardized proxy solutions in future iterations.

4.1.4 Data Plane

The data plane comprises the physical or virtual infrastructure units that execute workloads. Consistent with cloud-native principles, these units are homogeneous, ephemeral, and automatically managed no single node is considered reliable, but the collective ensemble delivers high availability.

Agent

Each node in the data plane runs a lightweight agent process, whose responsibilities include:

- Periodically polling the control plane's REST API to retrieve the latest

desired configuration

- Comparing the desired state with the current local state
- Applying necessary changes (e.g., starting/stopping containers, updating network rules)
- Reporting node health and workload status back to the control plane

The agent embodies the Pull model and enables self-healing: if a node reboots or is replaced, the agent automatically re-synchronizes with the control plane, ensuring eventual consistency.

Workload Execution

Workloads are executed in containers to guarantee portability, isolation, and reproducibility across environments. The system uses the Docker Engine as its default container runtime due to its ubiquity, rich ecosystem, and mature tooling. However, the architecture is runtime-agnostic: the agent interfaces with the container runtime via a pluggable abstraction layer (e.g., using the Docker-compatible HTTP API). This design permits future migration to alternative runtimes such as Podman (rootless, daemonless) or CRI-O (Kubernetes-optimized) without modifying core orchestration logic.

Container images are pulled from a secure, authenticated registry, and all runtime configurations (CPU, memory, network) are derived from the declarative specifications stored in the control plane.

This layered, decoupled architecture directly addresses the functional and non-functional requirements:

- Fault tolerance via agent-driven self-healing (NFR1)
- Control-plane resilience through stateless design and PostgreSQL durability
- Scalability via homogeneous data-plane units and REST-based coordination
- Security via mTLS and container isolation

4.1.5 Architecture Finalization

Having examined each constituent component of the proposed system the control plane, data plane, agent, Edge ingress layer, and their interconnections it is now possible to consolidate these elements into a coherent and implementable architectural blueprint. The resulting design demonstrates strong alignment with both the functional and non-functional requirements established in Section 3.5.

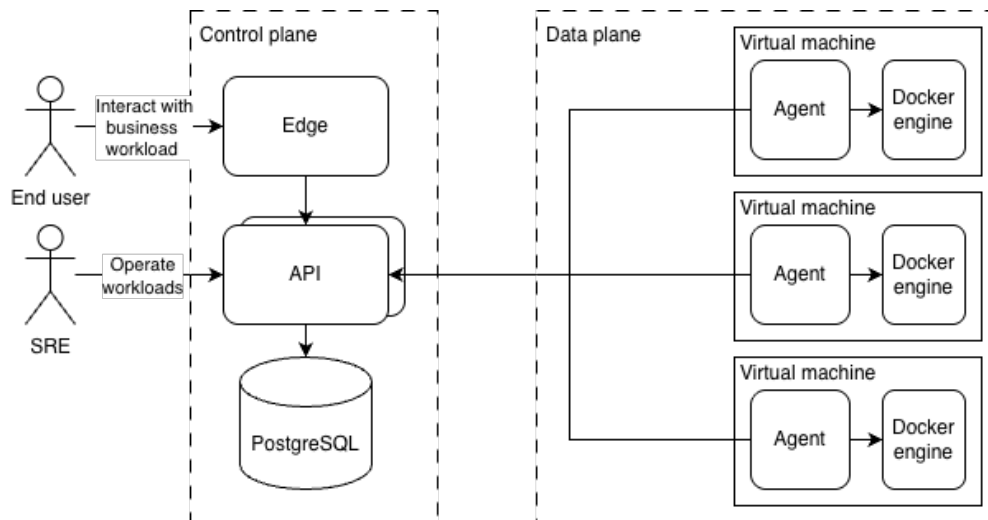


Fig. 4.3. Detailed component diagram of the finalized system architecture, illustrating interactions between the SRE client, control plane (with PostgreSQL backend), data-plane nodes (each running an agent and containerized workloads), and the Edge ingress component.

The design deliberately avoids over-engineering by leveraging lightweight,

well-understood technologies (PostgreSQL, REST, Docker) while retaining extensibility for instance, through pluggable runtimes or future integration with mature proxies like Envoy. This balance between simplicity and capability ensures the prototype remains feasible for implementation within the scope of this research, yet representative of real-world operational constraints.

Figure 4.3 presents a comprehensive view of the finalized architecture, capturing data flows, component responsibilities, and communication protocols. This blueprint serves as the definitive reference for the subsequent implementation phase and provides the structural foundation against which the prototype's correctness and performance will be validated.

4.2 Implementation

4.2.1 Technical Scenarios

This section presents the concrete implementation details of the system, illustrating how the pull-based orchestration model operates in practice through key technical scenarios. Each scenario demonstrates a core aspect of the system's functionality, including workload submission, agent discovery, lease management, failure recovery, and ingress routing.

Scenario 1: Workload Creation and Submission

The initial entry point for orchestrating a workload is through the command-line interface tool `mctl`. Operators define workload specifications in a declarative YAML format that encapsulates the container image, resource requirements, command invocation, environment variables, and optional port mappings. Figure 4.4

presents an example workload specification for a long-running HTTP server.

```
image: "python:3.9"
command:
  - "python"
  - "-m"
  - "http.server"
  - "8000"
cpu: 200          # 200 millicores (0.2 CPU cores)
ram: 512          # 512 megabytes
env:
  PYTHONUNBUFFERED: "1"
container_port: 8000
```

Fig. 4.4. Example workload specification in YAML format for a Python HTTP server. The `cpu` field specifies resource requirements in millicores, while `ram` is expressed in megabytes.

The submission workflow begins when the operator executes `mctl apply workload -f workload.yaml`. The CLI tool parses the YAML specification and issues a POST request to the control plane's REST API endpoint `/api/v1/workloads`. Upon receiving the request, the control plane validates the specification, generates a unique identifier (UUIDv4) for the workload, and persists the workload record in PostgreSQL with an initial status of `new`. The system returns the workload identifier to the operator, enabling subsequent tracking and management operations.

Figure 4.5 illustrates the complete workload creation sequence.

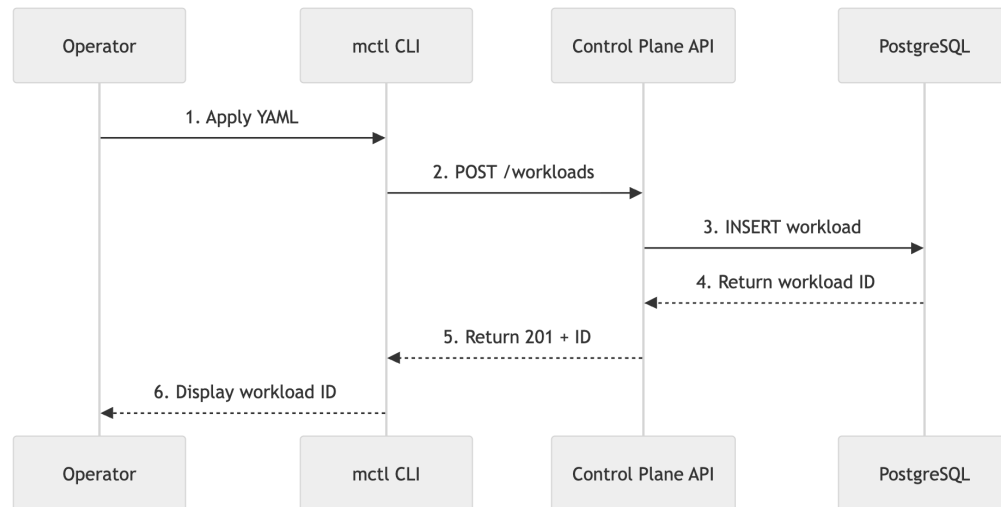


Fig. 4.5. Sequence diagram illustrating the workload creation and submission workflow. The operator submits a YAML specification via mctl, which is validated and persisted by the control plane, returning a unique workload identifier.

Scenario 2: Agent Discovery and Workload Acquisition

The pull-based model manifests most clearly in how agents discover and acquire workloads. Each agent runs a periodic reconciliation loop, orchestrated by a generic supervisor pattern that executes registered tasks at configurable intervals. By default, the supervisor triggers the workload acquisition logic every 10 seconds.

During each iteration, the agent performs the following sequence:

- (1) The agent queries the control plane's `GET /api/v1/workloads` endpoint, passing its available resource capacity as query parameters (e.g., `cpu=4000` and `ram=8192`). The control plane responds with a list of workloads whose resource requirements fit within the agent's capacity and which do not

currently have active leases.

- (2) The agent filters the response to exclude workloads it has already acquired, maintaining a local cache of owned workloads to prevent duplicate acquisition.
- (3) If candidate workloads remain, the agent selects one (using a first-fit strategy) and attempts to acquire a lease by issuing a
`PUT /api/v1/workloads/{id}/lease?node_id={node_id}`
request to the control plane.
- (4) The control plane records the lease in the database, associating the workload identifier with the requesting node's identifier, along with a timestamp. The lease acquisition is atomic: if multiple agents simultaneously attempt to lease the same workload, the database's uniqueness constraint ensures only one succeeds.

Once the lease is acquired, the agent transitions the workload locally to the pending status and initiates the container creation process using the Docker Engine API.

Figure 4.6 depicts the workload acquisition sequence with lease management.

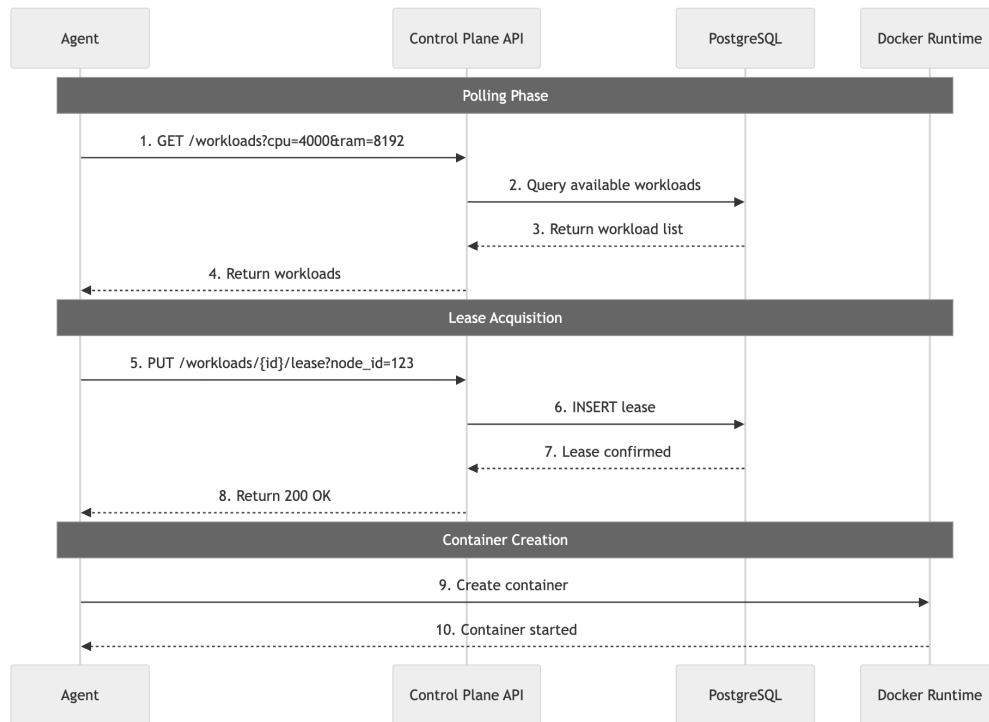


Fig. 4.6. Sequence diagram illustrating agent-driven workload acquisition. The agent polls for available workloads, acquires a lease, and creates the container via Docker.

Scenario 3: Container Lifecycle Management

Upon successfully leasing a workload, the agent invokes the container runtime to create and start the container according to the specification. The agent translates the abstract workload specification into Docker-specific configuration, setting CPU and memory limits using cgroups via Docker's `-cpus` and `-memory` flags, passing environment variables, and exposing the specified container port.

The agent then enters a monitoring phase, during which it periodically checks the container's health status. If the container exits unexpectedly, the agent attempts to restart it, subject to a configurable retry policy. Throughout the container's lifetime, the agent renews the lease every 10 seconds by issuing subsequent PUT requests to the control plane, updating the lease's `updated_at` timestamp. This

heartbeat mechanism signals to the control plane that the workload remains under active management.

Scenario 4: Lease Expiration and Fault Tolerance

A critical aspect of the system's resilience is its handling of node failures. The control plane runs a background task every 30 seconds that identifies and expires stale leases. A lease is considered stale if its `updated_at` timestamp predates a configured threshold (default: 30 seconds). When a node fails unexpectedly for example, due to a VM crash, network failure, or agent process termination it ceases to renew its leases. Consequently, the background task detects the stale leases and deletes them from the database, thereby releasing the associated workloads.

Once a workload's lease expires, the workload becomes available for acquisition by other agents. The system tracks workload status transitions based on event history: if a workload remains in the pending state without a valid lease for more than 90 seconds, it is automatically marked as `stuck`, indicating a persistent failure to start. Conversely, if a workload in the `active` state loses its lease, it becomes eligible for rescheduling onto a healthy node.

This design embodies the pull model's self-healing property: failures are detected indirectly through the absence of lease renewals, and recovery occurs automatically as other agents discover and acquire the orphaned workloads. No explicit push-based notification or failure detection mechanism is required from the control plane.

Figure 4.7 illustrates the fault tolerance mechanism, including node failure detection and workload rescheduling.

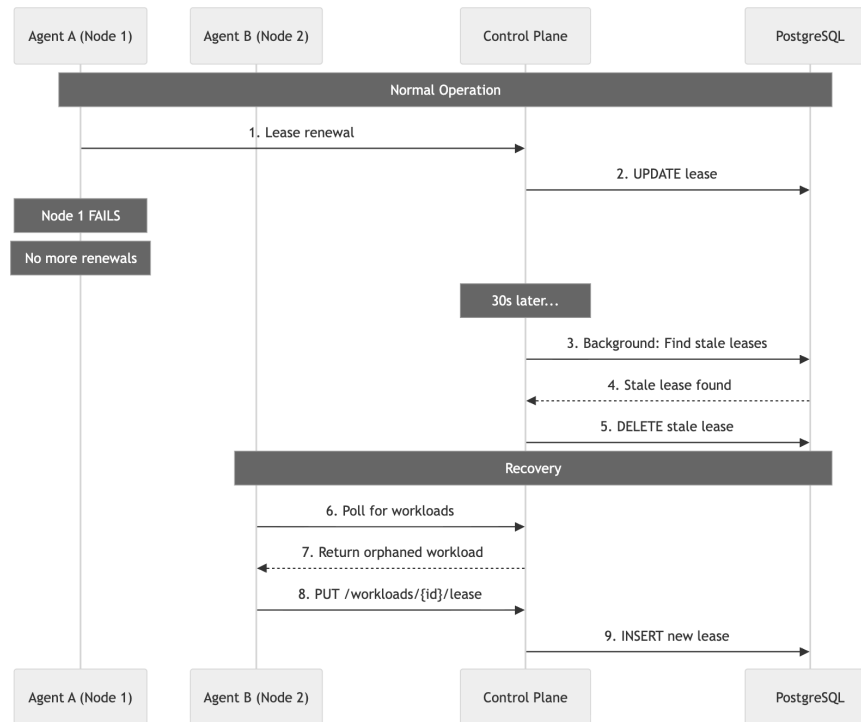


Fig. 4.7. Sequence diagram illustrating fault tolerance through lease expiration. When Agent A fails, the background task expires the stale lease after 30 seconds, enabling Agent B to acquire and reschedule the workload.

Scenario 5: Event Propagation and Status Tracking

The system maintains an auditable history of state transitions through an event log. Agents generate events for significant lifecycle events: container creation, container exit, health check success, and health check failure. Each event encapsulates the workload identifier, node identifier, action type, and outcome status. Agents POST these events to the `/api/v1/events` endpoint, where the control plane persists them in the database.

The control plane uses event history to infer workload status. For example, a sequence of successful health check events indicates a healthy active workload, while repeated exit events with non-zero exit codes suggest a failed state. This event-driven status computation ensures that the system's view of workload

health derives from ground-truth observations from the data plane, rather than assumptions made by the control plane.

Figure 4.8 shows the event propagation flow from agent to control plane, including status inference.

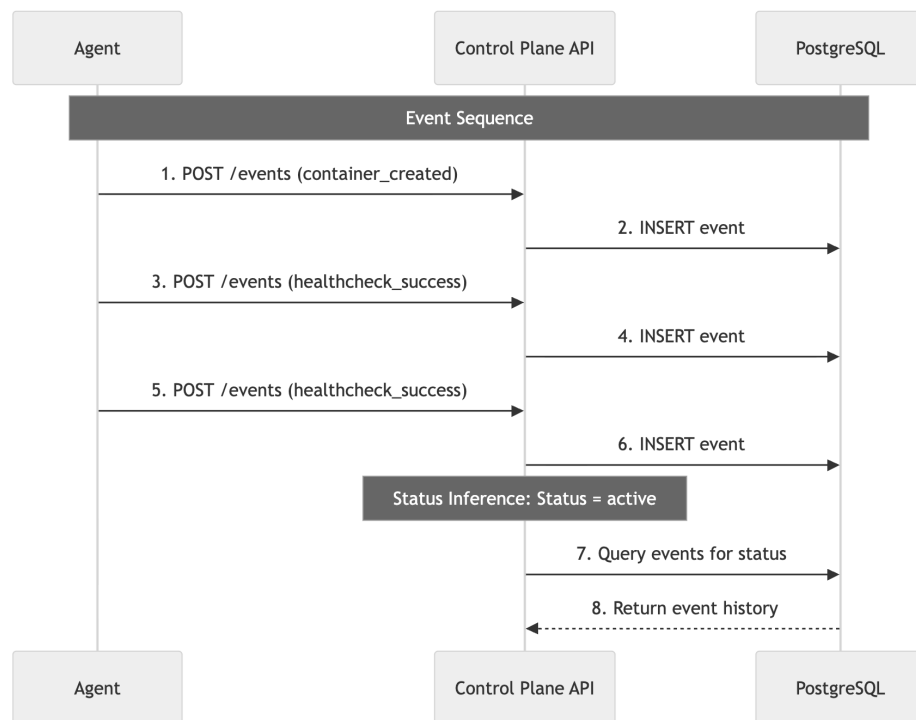


Fig. 4.8. Sequence diagram illustrating event propagation and status inference. Agents emit lifecycle events which are persisted in PostgreSQL; the control plane infers workload status from event history patterns.

Scenario 6: Ingress Routing via Edge Component

Workloads that expose HTTP endpoints require a unified access point for external clients. The Edge component addresses this need by periodically querying the control plane's GET `/api/v1/edges` endpoint to retrieve the current mapping of route prefixes to workload backend addresses. Each edge entry specifies a path prefix (e.g., `/my-service`) and an upstream address (e.g., `node-1:8000`).

The Edge component implements a reverse proxy that parses incoming HTTP requests, extracts the path prefix, and forwards the request to the corresponding backend using HTTP. This design enables operators to expose services through a single canonical endpoint while the actual workload instances may be scheduled on any node in the cluster. As workloads are rescheduled following failures, the Edge automatically updates its routing configuration upon the next periodic fetch, ensuring that traffic continues to flow to healthy instances without manual reconfiguration.

Implementation Architecture Summary

The implementation is organized into four main components, each adhering to a layered architecture pattern that separates concerns across domain, use case, infrastructure, and implementation layers:

- **Control Plane** (pkg/controlplane/): Stateless REST API server that exposes endpoints for workload management, lease operations, and event ingestion. Uses Uber FX for dependency injection and runs background tasks for lease expiration.
- **Agent** (pkg/agent/): Pull-based worker that periodically polls the control plane, manages container lifecycle via Docker, renews leases, and reports events. Employs a supervisor pattern for periodic task management.
- **Edge** (pkg/edge/): Network proxy that dynamically configures routing rules based on edge configurations fetched from the control plane.
- **mctl** (pkg/mctl/): Command-line interface for workload submission, querying, and management.

All components communicate exclusively via REST over HTTPS, with the control plane maintaining PostgreSQL as the authoritative data store. The architecture is horizontally scalable: multiple control plane instances can run behind a load balancer, and agents are stateless and homogeneous, enabling elastic scaling of the data plane.

Figure 4.9 presents a comprehensive end-to-end sequence demonstrating the complete lifecycle from workload creation through node failure to automatic recovery.

This implementation demonstrates that a pull-based orchestration system can be constructed with minimal operational complexity while retaining the core benefits of self-healing, fault tolerance, and decentralized coordination. The subsequent evaluation chapter presents empirical measurements of the system's performance, including recovery time objective achievement and resource efficiency.

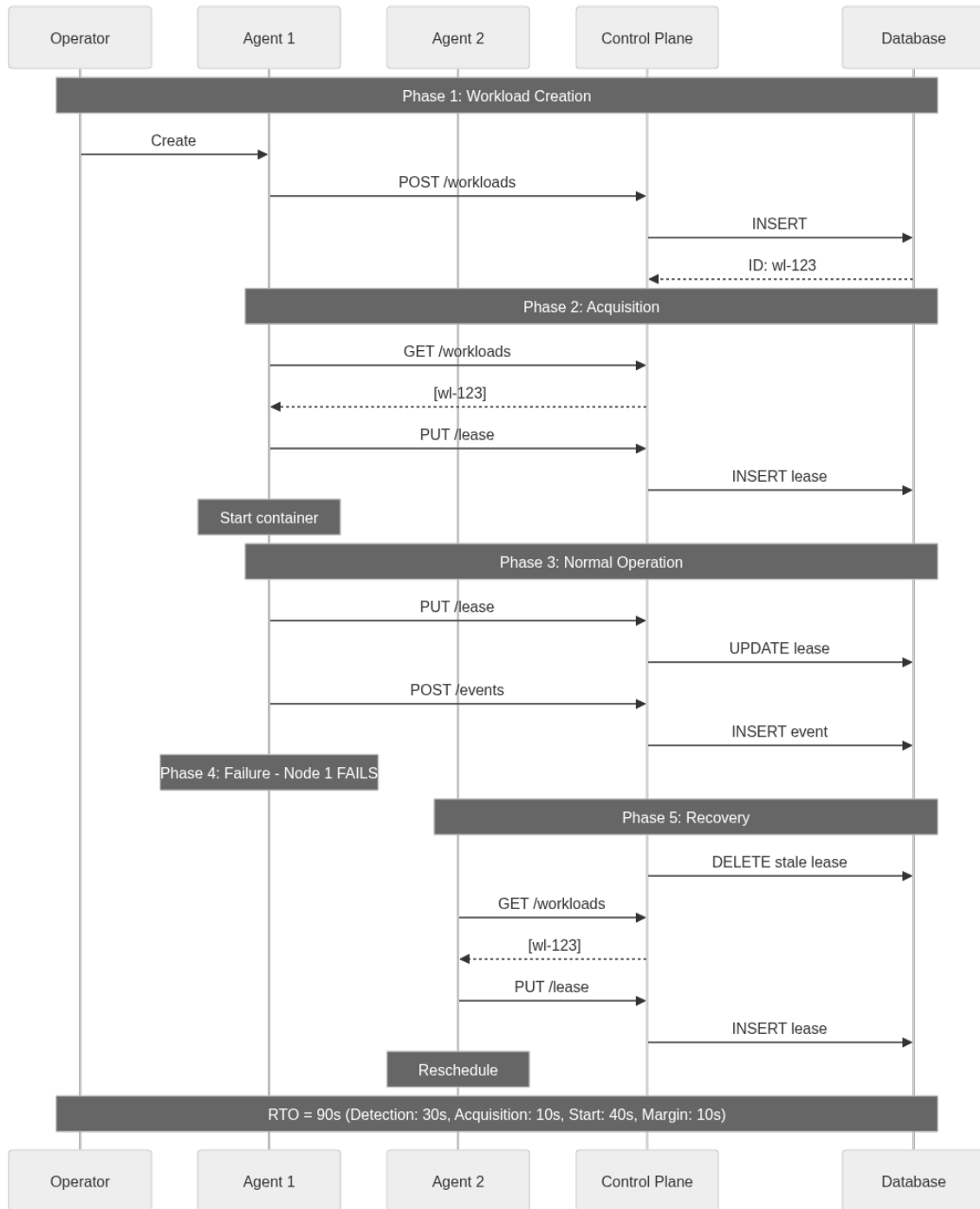


Fig. 4.9. Comprehensive end-to-end sequence demonstrating the complete pull-based orchestration lifecycle: workload creation, agent acquisition, normal operation with lease renewal, node failure detection through lease expiration, and automatic recovery by another agent. The annotated RTO box shows how the 90-second Recovery Time Objective is achieved.

Chapter 5

Evaluation

5.1 Methodology

5.1.1 Test Environment

The evaluation was conducted on real distributed infrastructure provided by Selectel, ensuring representative network conditions and resource constraints similar to production deployments. The test infrastructure consisted of:

- **Infrastructure Provider:** Selectel cloud platform
- **Architecture:** 3 nodes (1 control plane, 2 agents)
- **Node Configuration:** Each node configured with 2 vCPU and 4 GB RAM
- **Platform:** AMD64 architecture
- **Control Plane:** Deployed and exposed on port 8080
- **Agent Nodes:** 2 agents deployed on separate nodes, each configured with resource reservations for CPU and memory

- **Container Runtime:** Docker Engine for container lifecycle management
- **Database:** PostgreSQL 16 hosting the application schema with all migrations applied

5.1.2 Test Scenarios

Four comprehensive test scenarios were developed to validate system functionality and measure key performance metrics. To establish strong statistical validity, scenarios were executed with multiple trials: **BasicWorkloadAcquisition** (200 trials), **MultipleWorkloads** (100 trials), **Rescheduling** (100 trials), and **ConcurrentWorkloads** (10 trials).

1. **BasicWorkloadAcquisition:** Single workload creation and scheduling to verify basic orchestration functionality
2. **MultipleWorkloads:** Two workloads created simultaneously to test concurrent scheduling
3. **Rescheduling:** Agent failure simulation to measure recovery time and automatic rescheduling
4. **ConcurrentWorkloads:** Multiple workloads scheduled on a single node to test resource management

All tests followed a black-box approach, interacting exclusively through the control plane API and verifying expected state transitions in the database and container runtime.

5.1.3 Metrics Definition

Two primary metrics were measured to evaluate system performance:

- **Service Level Objective (SLO):** Time from workload creation request to the control plane marking the workload status as active. This metric measures how quickly the system can schedule and initialize new workloads.
- **Recovery Time Objective (RTO):** Time from node failure (process termination) to the control plane marking the workload status as active on a different node. This metric measures the system's ability to recover from node failures.

The active status is used as the completion criterion because it indicates that the control plane has confirmed successful workload initialization, representing the end-to-end scheduling latency including lease acquisition, container creation, and status reporting.

5.2 Test Results

5.2.1 Service Level Objective (SLO)

Table 5.1 presents the SLO measurements across different workload scenarios, including percentile values and 95% confidence intervals.

The single workload scenario demonstrates that the system can schedule individual workloads with an average time of 15.6 seconds (95% CI: [15.0s, 16.2s]). The P95 latency of 24.6 seconds indicates that 95% of workloads are scheduled within this bound, providing a reliable upper estimate for capacity planning.

Scenario	Min	Max	Avg	Median	StdDev	P90	P95	P99	CI Low	CI High	Trials
Single workload	10.5s	29.7s	15.6s	14.2s	4.1s	23.7s	24.6s	27.6s	15.0s	16.2s	200
Two workloads	11.6s	49.9s	17.9s	19.5s	4.9s	21.1s	21.4s	22.7s	17.0s	18.9s	99

TABLE 5.1

Service Level Objective (SLO) measurements with confidence intervals and percentiles

When scheduling two workloads simultaneously, the average SLO increases to 17.9 seconds (95% CI: [17.0s, 18.9s]). The P95 of 21.4 seconds shows that the typical concurrent scheduling overhead is modest, with a P99 of 22.7 seconds indicating that nearly all concurrent scheduling operations complete within this bound.

A Mann-Whitney U test confirms that the difference between single and concurrent workload SLO distributions is statistically significant ($U = 7478.5$, $p = 0.0006$, $p < 0.05$), indicating that concurrent scheduling introduces a measurable but bounded performance overhead.

Figure 5.1 visualizes the distribution of SLO measurements across scenarios, highlighting the concentration of values below 25 seconds and the presence of outliers in the concurrent scenario.

5.2.2 Recovery Time Objective (RTO)

Table 5.2 presents the RTO measurements from node failure scenarios.

Metric	Min	Max	Avg	Median	StdDev	P90	P95	P99	CI Low	CI High	Trials
RTO (node failure)	27.0s	66.4s	47.8s	48.9s	10.2s	57.9s	59.3s	60.0s	45.8s	49.8s	100

TABLE 5.2

Recovery Time Objective (RTO) measurements with confidence intervals and percentiles

The RTO measurements demonstrate that the system can recover from node failures with an average time of 47.8 seconds (95% CI: [45.8s, 49.8s]). The

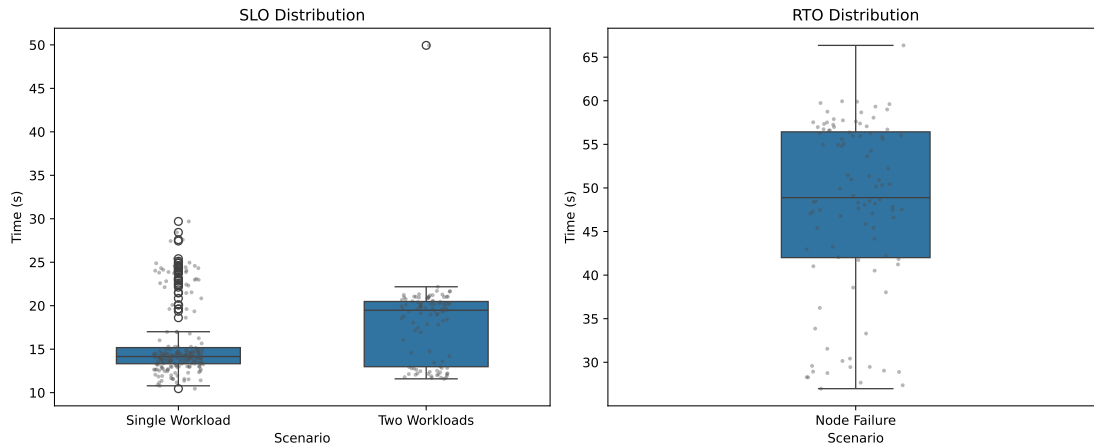


Fig. 5.1. SLO and RTO distribution comparison. Left: SLO for single and concurrent workload scenarios. Right: RTO distribution for node failure recovery. Individual data points are shown as scatter overlay.

narrow confidence interval (± 2.0 seconds) indicates highly predictable recovery behavior. The P95 of 59.3 seconds provides a reliable upper bound: 95% of node failures result in workload recovery within one minute.

The RTO is composed of three phases: (1) lease expiration detection (30 seconds), (2) backup agent polling and lease acquisition (2–20 seconds), and (3) container creation and status reporting (approximately 10–15 seconds). The standard deviation of 10.2 seconds indicates moderate variance in recovery times, primarily driven by differences in container startup times and polling alignment.

All 100 trials resulted in successful workload recovery, demonstrating that the lease-based failure detection mechanism operates reliably.

Figure 5.2 presents the cumulative distribution function for all measured metrics, showing that SLO values converge rapidly while RTO exhibits a more gradual slope due to the fixed 30-second lease timeout component.

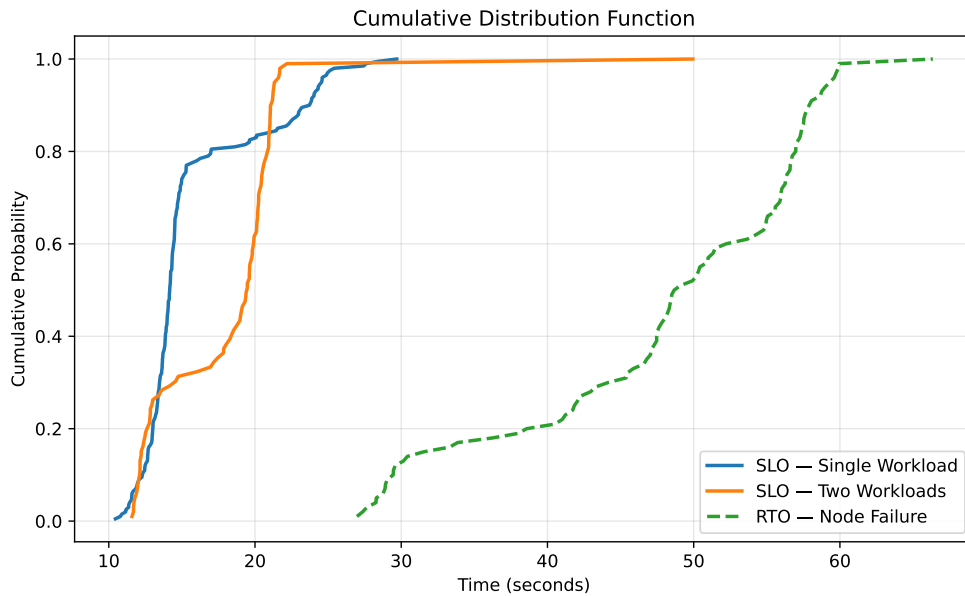


Fig. 5.2. Cumulative Distribution Function (CDF) of SLO and RTO measurements. The CDF shows the probability that a measurement falls at or below a given time value.

5.2.3 Functional Validation

Beyond quantitative metrics, the tests validated core functional requirements:

- **Distributed Operation:** Multiple agents successfully coordinated workload assignment without conflicts. The pull-based model with lease exclusivity prevented duplicate workload acquisitions across agents.
- **Automatic Rescheduling:** All 100 trials of the rescheduling scenario resulted in successful workload recovery on a different node without manual intervention. The lease expiration mechanism correctly detected node failures and triggered re-acquisition by backup agents.
- **Concurrent Workload Handling:** The system successfully handled multiple simultaneous workload creations without resource exhaustion or race conditions. Database transaction isolation ensured correct serialization of

concurrent lease acquisition requests.

- **Lease Exclusivity:** Each workload was acquired by exactly one agent, with no double-acquisition or lease conflicts observed across 400 total workload placements across all test scenarios.

5.3 Discussion

5.3.1 Performance Analysis

The test results demonstrate that the pull-based orchestration model achieves predictable scheduling performance with acceptable latency for short-lived workloads. The average SLO of 15.6 seconds (95% CI: [15.0s, 16.2s]) for single workloads and 17.9 seconds (95% CI: [17.0s, 18.9s]) for concurrent workloads is suitable for batch processing, CI/CD pipelines, and other use cases where workload initialization time is not critical compared to total execution time.

The RTO measurements confirm that the lease-based failure detection mechanism provides reliable fault tolerance with a tight confidence interval (95% CI: [45.8s, 49.8s]). While the 30-second lease timeout introduces a lower bound on failure detection, this trade-off simplifies the architecture by eliminating complex heartbeat protocols and consensus algorithms.

The standard deviation values (4.1 seconds for single SLO, 10.2 seconds for RTO) indicate low to moderate variance. The deterministic nature of the lease expiration mechanism ensures consistent failure detection performance, contributing to the overall predictability of the system.

5.3.2 Resource Efficiency

The control plane was configured with 0.5 vCPU and 1 GB RAM resource limits, representing a minimal footprint compared to existing orchestration platforms. This configuration successfully handled the tested workload (up to 2 concurrent workloads) without performance degradation or resource exhaustion.

Table 5.3 compares the control plane resource requirements of the proposed system with Kubernetes and HashiCorp Nomad.

Platform	Control Plane CPU	Control Plane Memory
Kubernetes	15 vCPU	22.5 GB
Nomad	3 vCPU	1.5–2.25 GB
Proposed System	0.5 vCPU	1 GB

TABLE 5.3

Control plane resource requirements for high-availability deployments. Kubernetes data from [1], Nomad data from [2].

The proposed system achieves a 97% reduction in CPU overhead and 96% reduction in memory overhead compared to Kubernetes. Relative to Nomad, the proposed system achieves 83% lower CPU overhead and 33–56% lower memory overhead. This efficiency stems from the pull-based architecture, which eliminates complex scheduling algorithms and cluster state management.

5.3.3 Design Trade-offs

The evaluation reveals fundamental trade-offs inherent in the pull-based orchestration model:

Simplicity vs. Feature Richness: The minimalist design yields substantial resource efficiency gains and operational simplicity, but sacrifices advanced

features such as sophisticated scheduling algorithms (bin-packing, affinity/anti-affinity), rolling updates, and comprehensive observability. For organizations with simple orchestration requirements, this trade-off is favorable; for complex enterprise deployments, the limitations may be prohibitive.

Detection Latency vs. Complexity: The 30-second lease timeout introduces bounded failure detection latency, but eliminates the need for complex heartbeat protocols and consensus-based failure detection. This trade-off is acceptable for use cases where rapid failure detection is not critical, but may be insufficient for mission-critical applications requiring sub-second recovery times.

Scheduling Optimality vs. Scalability: The first-come-first-served scheduling algorithm may yield suboptimal workload placement compared to global optimization algorithms, but enables linear scalability without scheduling bottlenecks. As the number of agents increases, the pull-based model avoids the scheduler bottleneck that limits push-based systems.

5.3.4 Threats to Validity

Several limitations affect the generalizability of the evaluation findings:

Scale Limitations: The evaluation tested up to 2 concurrent workloads across 2 agents with 100–200 trials per scenario. While this provides strong statistical significance for the tested scale (confirmed by narrow 95% confidence intervals), large-scale deployments with hundreds of workloads and dozens of agents may exhibit different performance characteristics. Database query performance and connection pool exhaustion become concerns at higher scale.

Workload Simplicity: Test workloads used minimal images (busybox) with simple command execution. Real-world applications with complex initialization

logic, dependency resolution, and health check endpoints may exhibit longer startup times and higher resource consumption.

Network Homogeneity: All nodes were deployed in the same Selectel cloud region with low-latency networking. Geographically distributed deployments with higher network latency may exhibit different performance characteristics, particularly for RTO measurements.

Test Duration: Each trial represents a single workflow execution. Long-running tests (hours or days) would be required to detect memory leaks, resource exhaustion, and other emergent behaviors that may not be apparent in short-duration tests.

5.4 Conclusion

This chapter presented empirical evaluation of the pull-based orchestration system, demonstrating predictable scheduling performance (SLO of 15.6 seconds average, 95% CI: [15.0s, 16.2s]) and reliable fault tolerance (RTO of 47.8 seconds average, 95% CI: [45.8s, 49.8s]) on real distributed infrastructure. Statistical analysis confirmed that concurrent scheduling introduces a measurable but bounded overhead (Mann-Whitney U test, $p = 0.0006$). Functional testing validated distributed operation, automatic rescheduling, and concurrent workload handling across 400 total workload placements.

The evaluation confirms that the pull-based architecture achieves substantial resource efficiency gains (97% CPU reduction, 96% memory reduction compared to Kubernetes) while maintaining acceptable performance for short-lived workloads. The minimalist design successfully trades feature richness for operational simplicity, making the system suitable for resource-constrained environments

and use cases where Kubernetes operational complexity is disproportionate to requirements.

The test results provide evidence that pull-based orchestration is a viable approach for small-scale deployments, offering a compelling alternative for organizations seeking simplicity over feature completeness. Future work should address scalability testing with larger workload counts, long-running stability tests, and performance characterization under network latency and partition scenarios.

Chapter 6

Conclusion

...

Bibliography cited

- [1] Kubernetes Authors, *Kubernetes documentation: Architecture overview*, <https://kubernetes.io/docs/concepts/architecture/>, Accessed: 31 October 2025, 2025.
- [2] HashiCorp, *Nomad documentation: Architecture overview*, <https://developer.hashicorp.com/nomad/docs/architecture>, Accessed: 31 October 2025, 2025.
- [3] M. Sebrechts, “Orchestrator conversation: Distributed management of cloud applications,” *International Journal of Network Management*, 2018.
- [4] D. Palma, M. Rutkowski, and T. Spatzier, “Tosca simple profile in yaml version 1.0,” OASIS, Committee Specification Draft 04 / Public Review Draft 01, 2015.
- [5] P. Rauba and N. Sontakos, “Self-healing machine learning: A framework for autonomous adaptation in real-world environments,” in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, Top-tier venue (NeurIPS), Author H-index: 5, 99 total citations, Curran Associates, 2024.
- [6] R. K. Vankayalapati, *Zero-trust security models for cloud data analytics: Enhancing privacy in distributed systems*, <https://papers.ssrn.com/sol3/>

- [papers.cfm?abstract_id=5121185](#), SSRN 5121185, 12 citations, Author total citations: 628 (Strong H-index), 2025.
- [7] B. Godin and J. P. Lane, “Pushes and pulls: History of the demand pull model of innovation,” *Science, Technology & Human Values*, 2013.
- [8] A. Dresch, “Design science research,” 2014, [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-07374-3_4.
- [9] E. Song *et al.*, “Canal mesh: A cloud-scale sidecar-free multi-tenant service mesh architecture,” in *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM, Aug. 2024. DOI: [10.1145/3651890.3672221](https://doi.org/10.1145/3651890.3672221). [Online]. Available: <https://dl.acm.org/doi/10.1145/3651890.3672221>.
- [10] M. Pantazoglou, G. Tzortzakakis, and A. Delis, “Decentralized and energy-efficient workload management in enterprise clouds,” *IEEE Transactions on Cloud Computing*, 2015.
- [11] Y. Wu *et al.*, “Task scheduling in geo-distributed computing: A survey,” *IEEE Internet Computing*, vol. 36, pp. 2073–2088, 2025. DOI: [10.1109/TPDS.2025.3591010](https://doi.org/10.1109/TPDS.2025.3591010). [Online]. Available: <https://ieeexplore.ieee.org/document/11086502>.
- [12] Google, *How google protects its production services*, <https://cloud.google.com/docs/security/production-services-protection>, Accessed: 31 October 2025, 2025.
- [13] M. Liaqat, “Federated cloud resource management: Review and discussion,” *Elsevier*, 2017.

- [14] D. Velez and P. Zlateva, "Cloud infrastructure security," *Open Research Problems in Network Security*, 2011.
- [15] H. Rasheed, "Data and infrastructure security auditing in cloud computing environments," *Elsevier*, 2014.
- [16] B. Burns, "Borg, omega, and kubernetes," *Communications of the ACM*, 2016.
- [17] B. Burns, *Kubernetes: Up & Running 3rd edition*. O Reilly, 2022.
- [18] K3S Maintainers, *K3s documentation*, <https://docs.k3s.io/>, Accessed: 8 January 2026, 2025.
- [19] K0S Maintainers, *K0s documentation*, <https://docs.k0sproject.io/>, Accessed: 8 January 2026, 2025.
- [20] D. job editors. "Sre engineers salaries at 2025." (2025), [Online]. Available: <https://dreamjob.ru/salary/sre-inzhener> (visited on 01/08/2026).
- [21] Stackoverflow Maintainers, *Stack overflow developer survey 2025*, <https://survey.stackoverflow.co/2025/>, Accessed: 8 January 2026, 2025.
- [22] Randy Bias, *The history of pets vs cattle and how to use the analogy properly*, <https://cloudscaling.com/blog/cloud-computing/the-history-of-pets-vs-cattle/>, Accessed: 10 January 2026, 2016.

Appendix A

Extra Stuff

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Appendix B

Even More Extra Stuff

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.